

Unit 2 - Recurrent Architectures and Sequence Modeling [12 Sessions]

- Recurrent neural networks (RNNs),
- LSTM,
- GRU,
- bidirectional models,
- Encoder-decoder architectures, attention mechanisms, and sequence generation tasks.

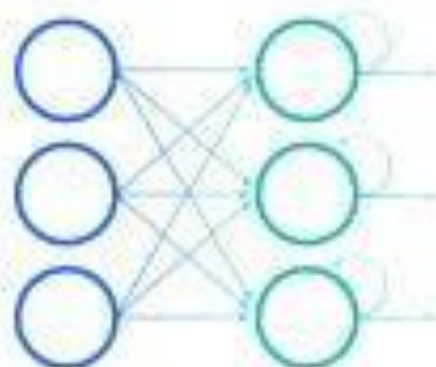
Recurrent neural networks (RNNs)

- A recurrent neural network or RNN is a deep neural network trained on sequential or time series data to create a machine learning (ML) model that can make sequential predictions or conclusions based on sequential inputs.
- An RNN might be used to predict daily flood levels based on past daily flood, tide and meteorological data. But RNNs can also be used to solve ordinal or temporal problems such as language translation, natural language processing (NLP), sentiment analysis, speech recognition and image captioning.

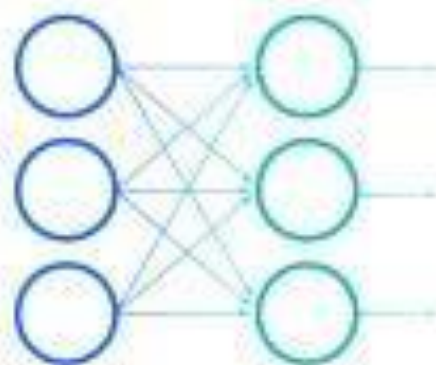
How RNNs work

- Recurrent neural networks use training data to learn. They are distinguished by their “memory” as they take information from prior inputs to influence the current input and output.
- While traditional deep learning networks assume that inputs and outputs are independent of each other, the output of recurrent neural networks depend on the prior elements within the sequence. While future events would also be helpful in determining the output of a given sequence, unidirectional recurrent neural networks cannot account for these events in their predictions.
- Another distinguishing characteristic of recurrent networks is that they share parameters across each layer of the network.
- Recurrent neural networks use forward propagation and backpropagation through time (BPTT) algorithms to determine the gradients (or derivatives), which is slightly different from traditional backpropagation as it is specific to sequence data.

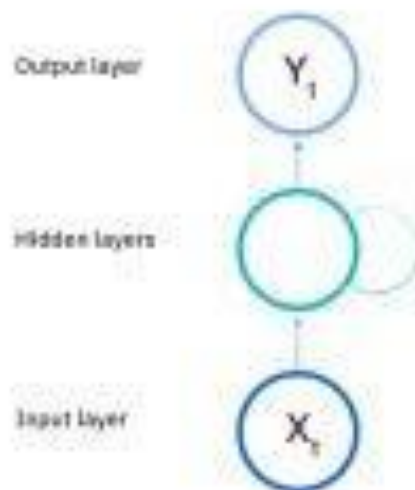
Recurrent Neural Networks



Feedforward Neural Networks

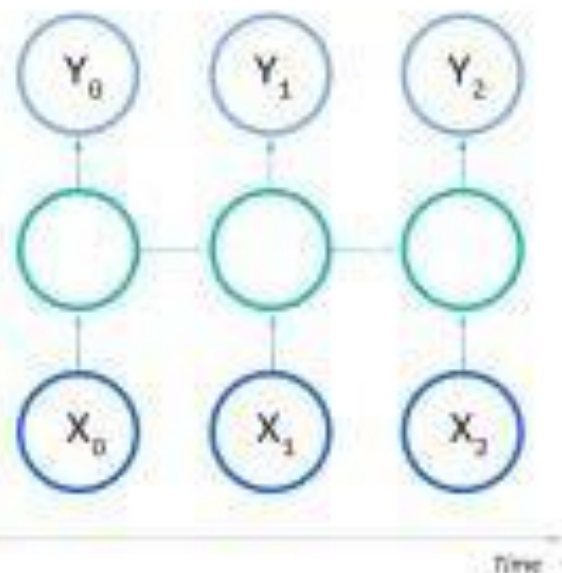


Rolled RNN



=

Unrolled RNN



Common activation functions

- An activation function is a mathematical function applied to the output of each layer of neurons in the network to introduce nonlinearity and allow the network to learn more complex patterns in the data.
- The activation function controls the magnitude of the neuron's output, keeping values within a specified range (for example, between 0 and 1 or -1 and 1), which helps prevent values from growing too large or too small during the forward and backward passes. In RNNs, activation functions are applied at each time step to the hidden states, controlling how the network updates its internal memory (hidden state) based on current input and past hidden states.

The Sigmoid Function is to interpret the output as probabilities or to control gates that decide how much information to retain or forget. However, the sigmoid function is prone to the vanishing gradient problem (explained after this), which makes it less ideal for deeper networks.

The Tanh (Hyperbolic Tangent) Function, which is often used because it outputs values centered around zero, which helps with better gradient flow and easier learning of long-term dependencies.

The ReLU (Rectified Linear Unit) might cause issues with exploding gradients due to its unbounded nature. However, variants such as Leaky ReLU and Parametric ReLU have been used to mitigate some of these issues.

Types of RNNs

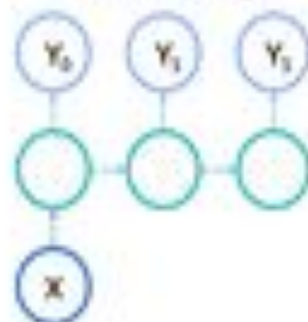
Feedforward networks map inputs and outputs one-to-one, and while we've visualized recurrent neural networks in this way in the diagrams before this, they do not have this constraint. Instead, their inputs and outputs can vary in length, and different types of RNNs are used for different use cases, such as music generation, sentiment classification and machine translation. Popular recurrent neural network architecture variants include:

- Standard RNNs
- Bidirectional recurrent neural networks (BRRNs)
- Long short-term memory (LSTM)
- Gated recurrent units (GNUs)
- Encoder-decoder RNN

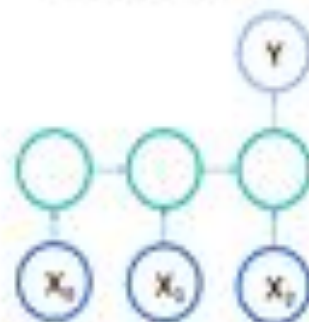
One-to-one



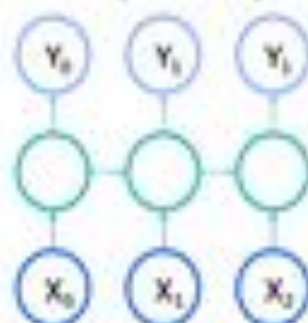
One-to-many



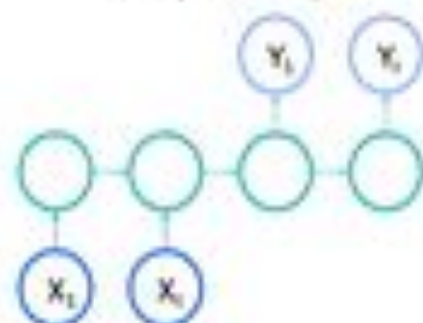
Many-to-one



Many-to-many



Many-to-many

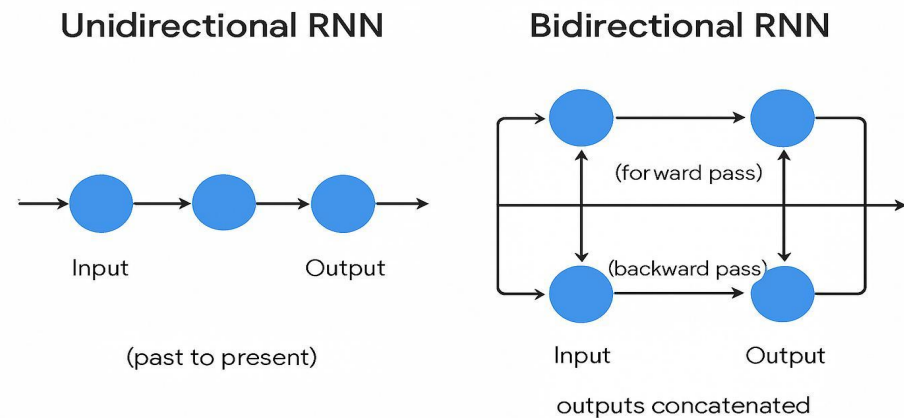


Standard RNNs

- The most basic version of an RNN, where the output at each time step depends on both the current input and the hidden state from the previous time step, suffers from problems such as vanishing gradients, making it difficult for them to learn long-term dependencies. They excel in simple tasks with short-term dependencies, such as predicting the next word in a sentence (for short, simple sentences) or the next value in a simple time series.
- RNNs are good for tasks that process data sequentially in real time, such as processing sensor data to detect anomalies in short time frames, where inputs are received one at a time and predictions need to be made immediately based on the most recent inputs.

Bidirectional recurrent neural networks (BRNNs)

While unidirectional RNNs can only be drawn from previous inputs to make predictions about the current state, bidirectional RNNs or BRNNs, pull in future data to improve the accuracy of it. Returning to the example of “feeling under the weather”, a model based on a BRNN can better predict that the second word in that phrase is “under” if it knows that the last word in the sequence is “weather.”



A **Bidirectional Recurrent Neural Network (BiRNN)** processes a sequence in **two directions**:

Forward RNN: Reads the sequence from **left to right**.

Backward RNN: Reads the sequence from **right to left**.

The outputs from both directions are combined to produce the final output.

Consider the sentence:

"I went to the bank to deposit my money."

Suppose we want to understand the meaning of the word **"bank"**.

The BiRNN processes the sentence as:

Forward Direction →

I → went → to → the → bank → to → deposit → my → money

Backward Direction ←

money → my → deposit → to → bank → the → to → went → I

The **forward RNN** understands the context before the word **"bank"**, such as **"went to the"**.

The **backward RNN** understands the context after **"bank"**, such as **"to deposit my money"**.

The two hidden states are combined:

Final representation of "bank" = Forward hidden state + Backward hidden state

Therefore, the network can correctly understand that **"bank"** refers to a **financial institution**, rather than a river bank.

Long short-term memory (LSTM)

- LSTM is a popular RNN architecture, which was introduced by Sepp Hochreiter and Juergen Schmidhuber as a solution to the vanishing gradient problem. This work addressed the problem of long-term dependencies. That is, if the previous state that is influencing the current prediction is not in the recent past, the RNN model might not be able to accurately predict the current state.
- As an example, let's say we wanted to predict the italicized words in, "Alice is allergic to nuts. She can't eat *peanut butter*." The context of a nut allergy can help us anticipate that the food that cannot be eaten contains nuts. However, if that context was a few sentences prior, then it would make it difficult or even impossible for the RNN to connect the information.
- To remedy this, LSTM networks have "cells" in the hidden layers of the artificial neural network, which have 3 gates: an input gate, an output gate and a forget gate. These gates control the flow of information that is needed to predict the output in the network. For example, if gender pronouns, such as "she", was repeated multiple times in prior sentences, you might exclude that from the cell state.

Consider the sentence:

"I grew up in India. I speak fluent Hindi."

Suppose the LSTM is processing the sentence word by word:

I → grew → up → in → India → I → speak → fluent → Hindi

When the LSTM reads "**India**", it stores this important information in its **cell state**. Later, when it processes "**I speak fluent Hindi**," it can use the previously stored context to understand the relationship between **India** and **Hindi**.

How LSTM Works

An LSTM uses three important gates:

Forget Gate: Decides what old information should be removed.

Input Gate: Decides what new information should be stored.

Output Gate: Decides what information should be passed to the next step

Simple Example of LSTM Memory

Input sequence:

The movie was not good, because the acting was boring.

When the LSTM reads the word "**not**", it can remember this information for later. When it reaches "**good**", the LSTM uses the stored context "**not**" to understand that the sentiment is negative.

Without LSTM: The model may forget the word "**not**".

With LSTM: The model can preserve important context over many time steps.

Gated recurrent units (GRUs)

- A GRU is similar to an LSTM as it also works to address the short-term memory problem of RNN models.

Instead of using a “cell state” to regulate information, it uses hidden states, and instead of 3 gates, it has

2: a reset gate and an update gate. Similar to the gates within LSTMs, the reset and update gates control

how much and which information to retain.

- Because of its simpler architecture, GRUs are computationally more efficient and require fewer parameters compared to LSTMs. This makes them faster to train and often more suitable for certain real-time or resource-constrained applications.

Consider the sentence:

"The movie was not good, but the acting was excellent."

The GRU processes the sentence word by word:

The → movie → was → not → good → but → the → acting → was → excellent

When the GRU encounters the word **"not"**, it uses its **memory mechanism** to retain this important context.

Later, it processes **"good"** while considering the previous information.

The GRU uses two gates:

Update Gate: Decides how much of the previous information should be **retained** and how much new information should be added.

Reset Gate: Decides how much of the **previous information should be forgotten** when processing the new input.

Suppose the GRU is processing:

"I did not like this movie."

"I did" → GRU stores the context.

"not" → The reset/update mechanisms help preserve the important negative context.

"like" → GRU considers the previous **"not"** information.

"movie" → The final hidden state represents the overall negative sentiment.

Thus, the GRU can understand that **"not like"** indicates a **negative sentiment**, even though the important word **"not"** appears before **"like."**

Encoder-decoder RNNs

These are commonly used for sequence-to-sequence tasks, such as machine translation. The encoder processes the input sequence into a fixed-length vector (context), and the decoder uses that context to generate the output sequence.

However, the fixed-length context vector can be a bottleneck, especially for long input sequences.

Text Summarization

Input:

Artificial Intelligence is transforming many industries by automating tasks and improving decision-making.

Encoder: Reads and understands the complete sentence.

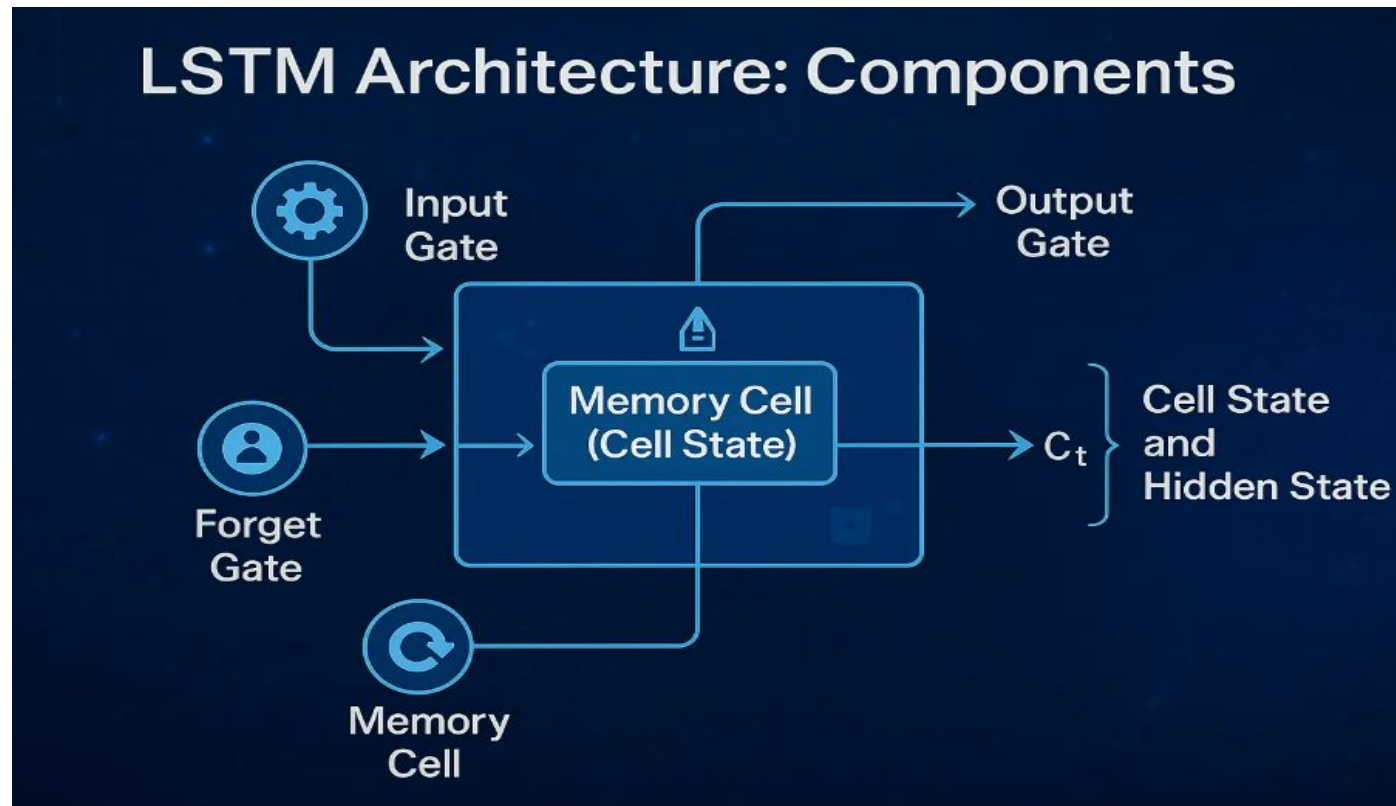
Decoder: Generates a shorter summary:

Output:

AI is transforming industries.

LSTM

LSTM (Long Short-Term Memory) networks are an extension of RNNs, with a more complex architecture to help the network learn what to remember, what to forget, and what to output over more extended sequences.



1. Memory Cell (Cell State):

The memory cell is the epicenter of the LSTM unit. A conveyor belt transports information across time steps with minimal alterations. The memory cell allows LSTM to store information for long intervals, making it feasible to capture long-term dependencies.

2. Input Gate:

The input gate controls the entry into the memory cell of new information. It applies a sigmoid activation function to determine which values will be updated and a tanh function to generate a candidate vector. This gate makes it possible to store only relevant new information.

3. Forget Gate:

This gate determines what should be thrown out of the memory cell. It gives values between 0 and 1; 0: “completely forget”, 1: “completely keep”. This selective forgetting is essential in avoiding memory overload.

4. Output Gate:

The output gate decides what piece in the memory cell goes to the next hidden state (and maybe even as output). It supports the network in determining which information from the current cell state would influence the next step along the sequence.

Cell State and Hidden State:

1. **Cell State (C_t):** It carries long-term memory modified by input and forget gates.
2. **Hidden State (h_t):** Represents the output value of the LSTM unit in a particular time step, which depends upon both the cell state and the output gate. It is transferred to the next LSTM unit and tends to be used in the final prediction.

How do These Components Work Together?

The LSTM unit performs the sequence of operations in every time step:

1. **Forget:** The forget gate uses the previous hidden state and current input to determine information to forget from the cell state.
2. **Input:** The input gate and the candidate values determine what new information needs to be added to the cell state.
3. **Update:** The cell state is updated when old retention information is merged with the chosen new input.
4. **Output:** The output gate will use the updated cell state to produce the next hidden state that will control the next step, and might be the output itself.

This complex gating system enables LSTMs to keep a well-balanced memory, which can retain critical patterns and forget unnecessary noise that traditional RNNs find difficult.

LSTM Algorithm: How It Works

1. Input at Time Step :

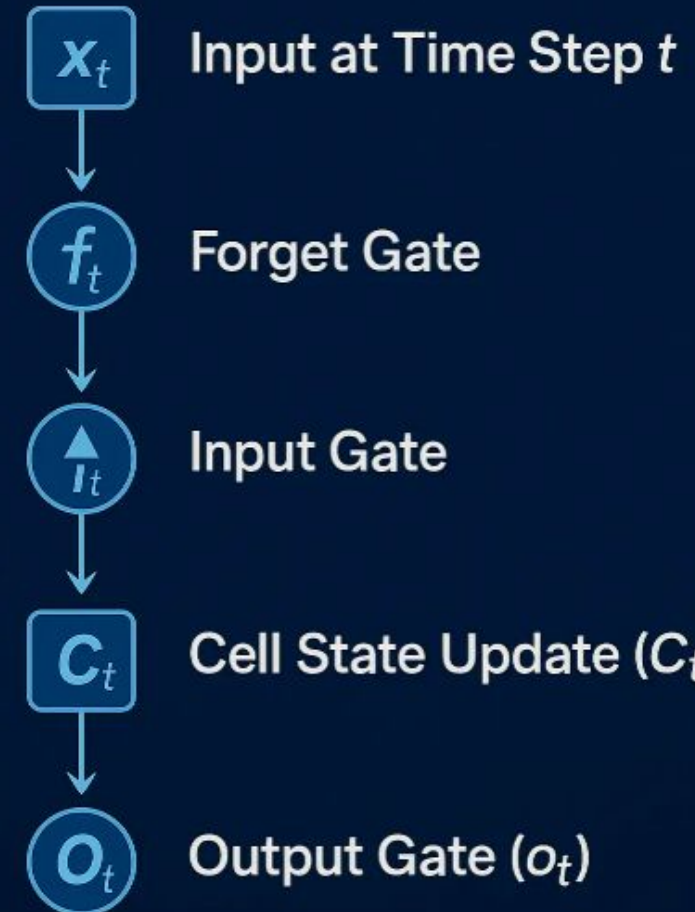
At each time step t , the LSTM receives two pieces of information:

- x_t : The current input to the LSTM unit (e.g., the next word in a sentence, or the next time value in a sequence)
- h_{t-1} : The previous hidden state carries the prior time step information.
- C_{t-1} : The previous cell state carries long-term memory from prior time steps.

2. Forget Gate (f_t):

The forget gate decides what information from the previous cell state should be discarded. It looks at the current input x_t and the last hidden state h_{t-1} and applies a **sigmoid function** to generate values between 0 and 1. 0 means “forget completely,” and 1 means “keep all information.”

LSTM Algorithm: How It Works



$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Where:

- f_t = Forget Gate vector
- σ = Sigmoid activation function
- W_f = Weight matrix for the forget gate
- h_{t-1} = Previous hidden state
- x_t = Current input
- b_f = Bias vector

The sigmoid function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The output f_t lies between **0 and 1**:

- **0** → completely forget the information
- **1** → completely retain the information
- **Between 0 and 1** → partially retain the information

For the cell state update, the forget gate is used as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Input Gate (iti_tit):

The input gate determines what new information should be added to the cell state. It has two components:

- The sigmoid layer decides which values will be updated (output between 0 and 1).
- The tanh layer generates candidate values for new information.
- Formula:

The **Input Gate** controls how much **new information** should be added to the cell state.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

Where:

- i_t = **Input Gate output** at time t
- σ = **Sigmoid activation function**
- W_i = Weight matrix for the Input Gate
- h_{t-1} = Previous hidden state
- x_t = Current input
- b_i = Bias vector

The sigmoid function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The Input Gate value ranges from **0 to 1**:

- $i_t = 0 \rightarrow$ do not add the new information
- $i_t = 1 \rightarrow$ allow the new information completely
- $0 < i_t < 1 \rightarrow$ partially allow the information

Candidate Cell State

The Input Gate works together with the candidate cell state:

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

Then the cell state is updated as:

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

So, i_t is read as “**i sub t**”, meaning the **Input Gate at time step t** .

Cell State Update (CtC_tCt):

The cell state is updated by combining the previous C_{t-1} (modified by the forget gate) and the new information generated by the input gate. The forget gate's output controls how much of the previous cell state is kept, while the input gate's output controls how much new information is added.

- **Formula:** The **Cell State Update** determines the new memory of the LSTM cell by combining the **previous cell state**, **Forget Gate**, **Input Gate**, and **Candidate Cell State**.

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

Where:

- C_t = **New cell state** at time t
- C_{t-1} = **Previous cell state**
- f_t = **Forget Gate**
- i_t = **Input Gate**
- $\tilde{C}_t$ = **Candidate cell state**
- $\odot$ = **Element-wise multiplication**

In simple words

New Memory = Old Memory × Forget Gate + New Candidate Information × Input Gate

So:

1. $f_t \odot C_{t-1}$ → decides **what old information to keep**.
2. $i_t \odot \tilde{C}_t$ → decides **what new information to add**.
3. Both are added to produce the **new cell state** C_t .

1. Output Gate (oto_tot):

The output gate determines which information from the cell state should be output as the hidden state for the current time step.

The current input x_t and the previous hidden state h_{t-1} are passed through a **sigmoid function** to decide which parts of the cell state will influence the secret state. The **tanh function** is then applied to the cell state to scale the output.

The **Output Gate** determines how much information from the updated cell state should be passed to the hidden state/output.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

Where:

- o_t = **Output Gate** at time step t
- σ = **Sigmoid activation function**
- W_o = Weight matrix for the Output Gate
- h_{t-1} = Previous hidden state
- x_t = Current input
- b_o = Bias

The hidden state is then calculated as:

$$h_t = o_t \odot \tanh(C_t)$$

In simple words

Output Gate → decides what information from the current cell state C_t should be exposed as the output h_t .

- $o_t \approx 0$ → information is mostly blocked.
- $o_t \approx 1$ → information is mostly passed to the hidden state.

Training LSTM Networks

1. Data Preparation for Sequential Tasks

Proper data preprocessing is crucial for LSTM performance:

- **Sequence Padding:** Ensure all input sequences have the same length by padding shorter sequences with zeros.
- **Normalization:** Scale numerical features to a standard range (e.g., 0 to 1) to improve convergence speed and stability.
- **Time Windowing:** For time series forecasting, create sliding windows of input-output pairs to train the model on temporal patterns.
- **Train-Test Split:** Divide the dataset into training, validation, and test sets, maintaining the temporal order to prevent data leakage.

2. Model Configuration: Layers, Hyperparameters, and Initialization

- **Layer Design:** Begin with an LSTM layer [1] and finish with a Dense output layer. For complex tasks, layer stacking LSTM layers can be considered.
- **Hyperparameters:**
 - **Learning Rate:** Start with a value from $1e-4$ to $1e-2$.
 - **Batch Size:** Common choices are 32, 64, or 128.
 - **Number of Units:** Usually between 50 and 200 units per LSTM layer.
 - **Dropout Rate:** Dropout (e.g., 0.2 to 0.5) can solve overfitting.
- **Weight Initialization:** Use Glorot or He initialization of weights to initialize the initial weights to move faster towards convergence and reduce vanishing/exploding gradient risks.

3. Training Process

Knowing the basic elements of LSTM training

- Backpropagation Through Time (BPTT)- This algorithm calculates gradients by unrolling the LSTM over time to allow the model to learn sequential dependencies.
- Gradient Clipping: Clip backpropagator- gradients during backpropagation to a given threshold (5.0) to avoid exploding gradients. This helps in the stabilization of training, especially in deep networks.
- Optimization Algorithms- Optimizer can be chosen to be of Adam or RMSprop type, which adjust their learning rates and are suitable for training LSTM.

APPLICATIONS OF LSTM IN DEEP LEARNING

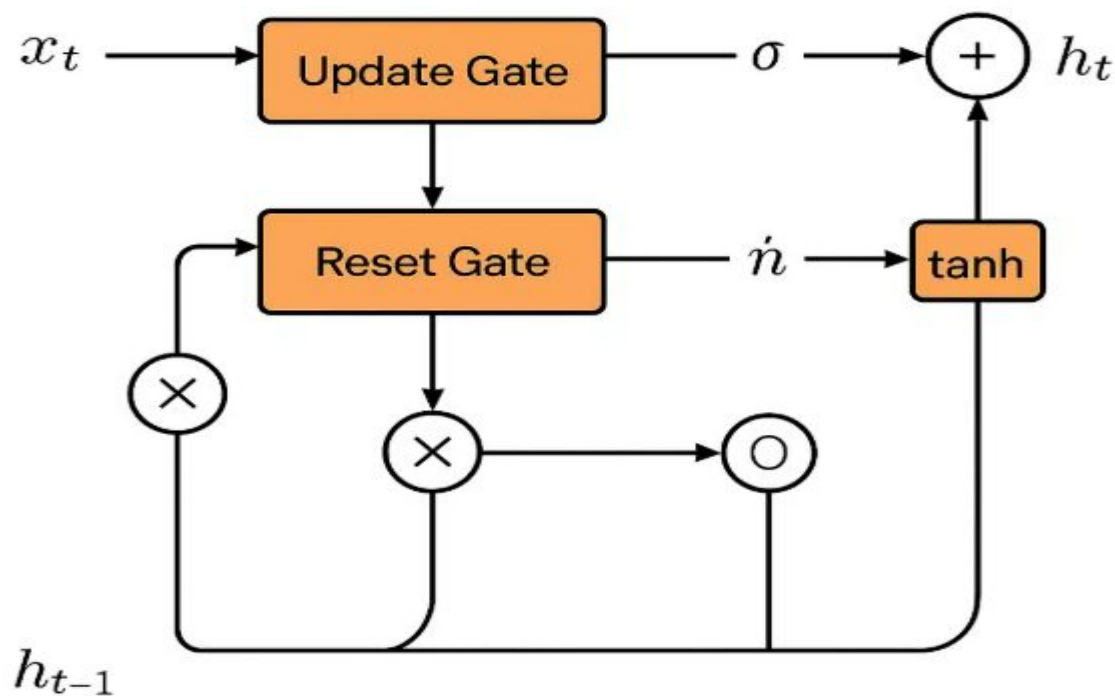
- Sequence Modeling
- Sentiment Analysis
- Speech Recognition
- Time Series Forecasting



Gated Recurrent Unit Networks (GRU) in Deep Learning

Gated Recurrent Units, or GRUs, are a type of [neural network](#) that helps computers make sense of sequences- things like sentences, time series, or even music. Unlike standard networks that treat each input separately, GRUs remember what came before, which is key when context matters.

Structure of Gated Recurrent Unit Networks



GRU uses two main gates:

- **Update Gate:** Controls how much past information is retained.
- **Reset Gate:** Controls how much previous information is forgotten when processing new input.

Information management: Gates help the GRU retain important information and ignore irrelevant or noisy data.

Combines old and new information: GRU intelligently blends previous hidden-state information with the current input.

Captures long-term dependencies: Helps learn patterns across sequences without being overwhelmed by unnecessary information.

Simpler than LSTM: GRU has **two gates**, whereas LSTM has **three gates** and a separate cell-state memory.

Fewer parameters: The simpler architecture generally requires fewer trainable parameters.

Faster training: Due to its simpler structure, GRUs are typically faster to train than LSTMs.

Good performance: GRUs can achieve performance comparable to LSTMs for many sequence-learning tasks.

Suitable for smaller datasets: They can be a practical choice when the dataset is not very large or complex.

Overall advantage: GRUs provide a **good balance between simplicity, efficiency, and ability to learn sequential patterns**.

GRU Architecture

The GRU architecture consists of the following components:

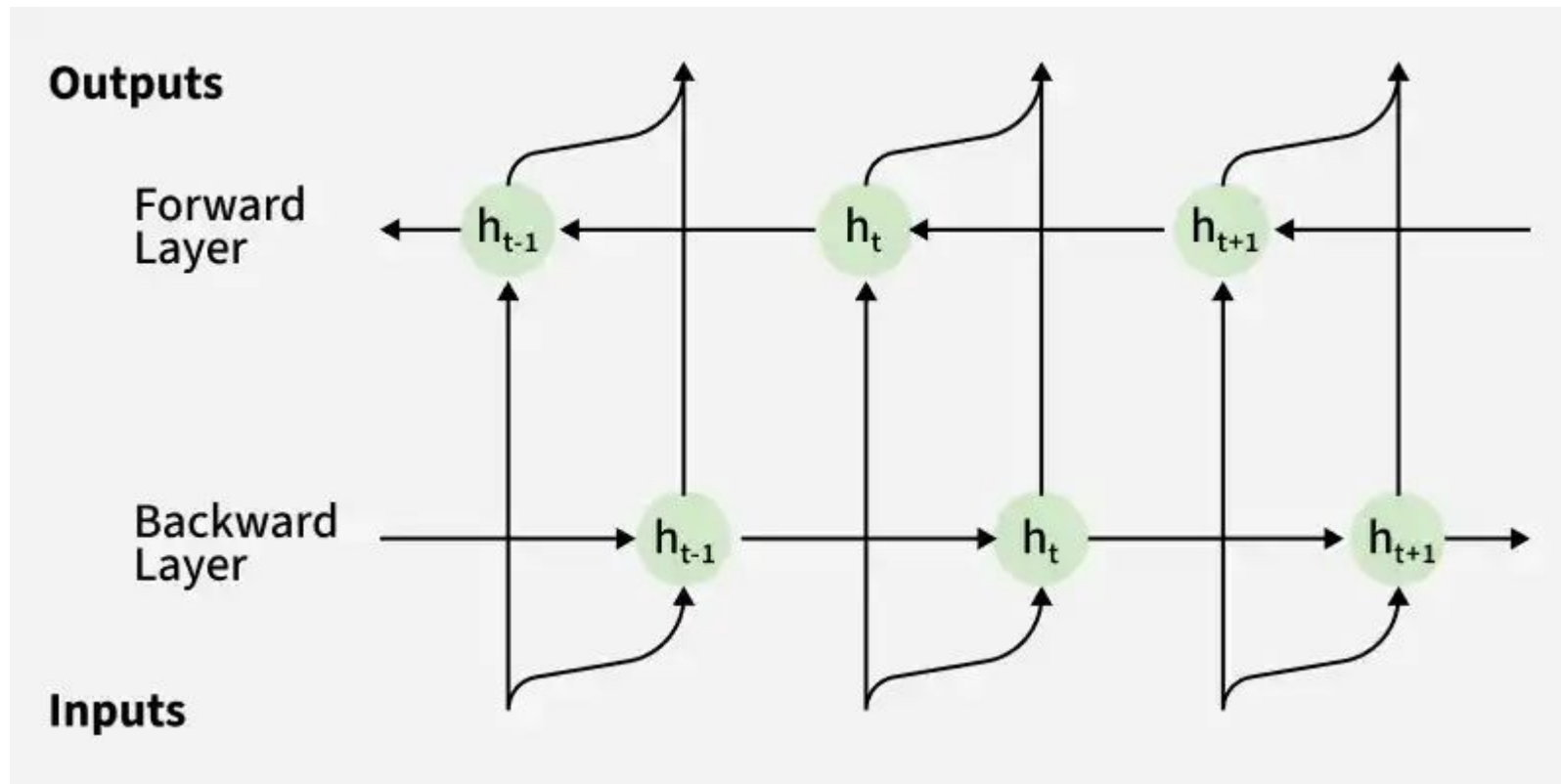
- 1. Input layer:** The input layer takes in sequential data, such as a sequence of words or a time series of values, and feeds it into the GRU.
- 2. Hidden layer:** The hidden layer is where the recurrent computation occurs. At each time step, the hidden state is updated based on the current input and the previous hidden state. The hidden state is a vector of numbers that represents the network's “memory” of the previous inputs.
- 3. Reset gate:** The reset gate determines how much of the previous hidden state to forget. It takes as input the previous hidden state and the current input, and produces a vector of numbers between 0 and 1 that controls the degree to which the previous hidden state is “reset” at the current time step.
- 4. Update gate:** The update gate determines how much of the candidate activation vector to incorporate into the new hidden state. It takes as input the previous hidden state and the current input, and produces a vector of numbers between 0 and 1 that controls the degree to which the candidate activation vector is incorporated into the new hidden state.

5. Candidate activation vector: The candidate activation vector is a modified version of the previous hidden state that is “reset” by the reset gate and combined with the current input. It is computed using a tanh activation function that squashes its output between -1 and 1.

6. Output layer: The output layer takes the final hidden state as input and produces the network’s output. This could be a single number, a sequence of numbers, or a probability distribution over classes, depending on the task at hand.

- **Bidirectional models**

Bidirectional Recurrent Neural Networks (BRNNs) are an advanced form of [RNNs](#) that process sequential data in both forward and backward directions. This allows the network to use both past and future context, improving understanding and prediction accuracy.



Working of Bidirectional Recurrent Neural Networks (BRNNs)

BRNNs process sequential data in both forward and backward directions to capture complete contextual information from a sequence.

Step 1: Input Sequence

A sequence of data points is provided as input, where each element is represented as a vector.

Step 2: Dual Direction Processing

The sequence is processed in two directions

- **Forward direction:** uses current input and previous hidden state
- **Backward direction:** uses current input and next hidden state

Step 3: Hidden State Computation

Hidden states are computed using weighted inputs and activation functions, allowing the network to retain sequence information.

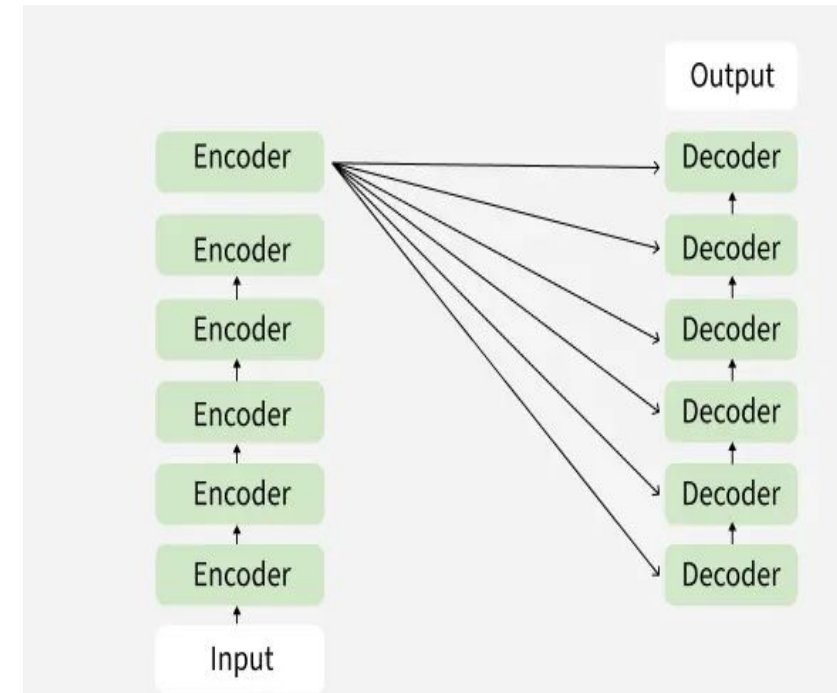
Step 4: Output Generation

The outputs are generated from the hidden states and can be used directly for prediction or passed to additional layers for further processing.

- **Encoder-decoder architectures**

The encoder-decoder model is a neural network used for tasks where both input and output are sequences, often of different lengths. It is commonly applied in areas like translation, summarization and speech processing.

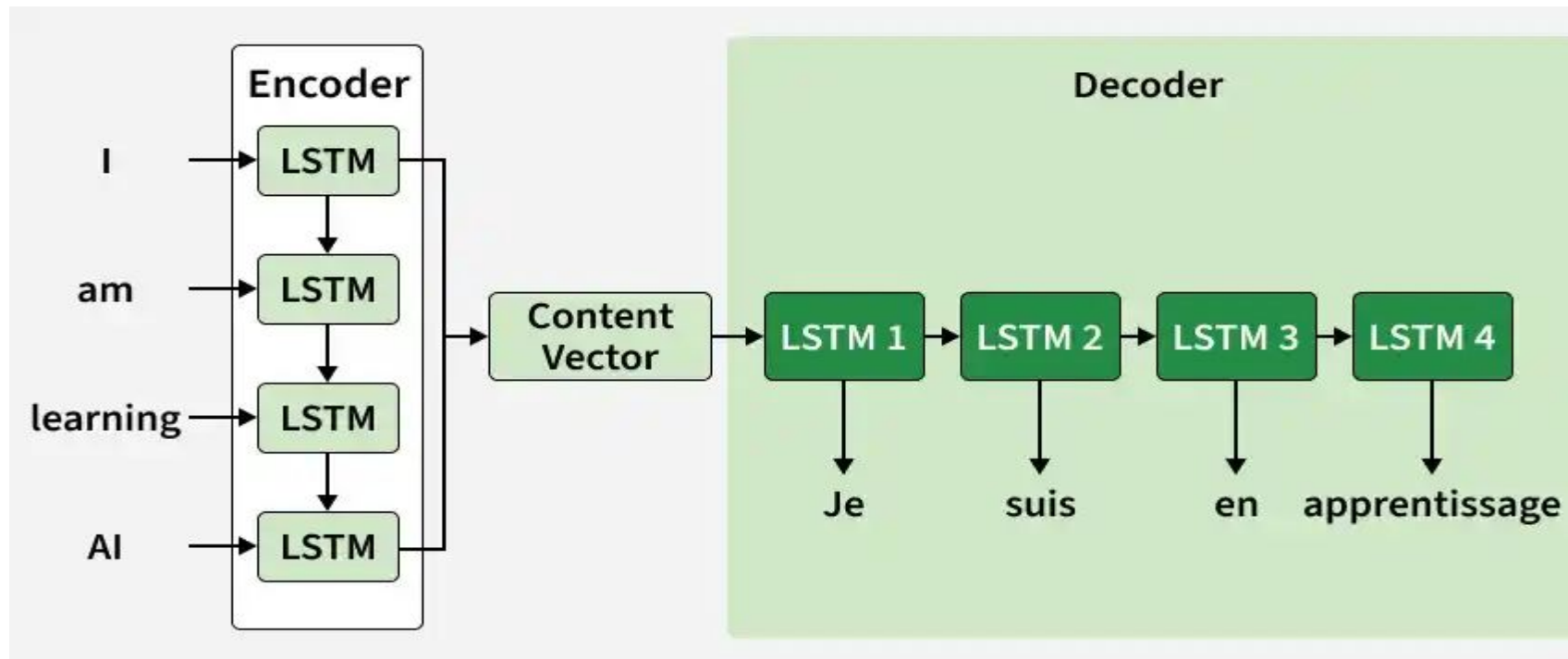
- The encoder processes the input sequence and converts it into a fixed representation (context vector)
- The decoder uses this representation to generate the output sequence step by step
- Works well for tasks where input and output lengths are different



Encoder

The encoder processes the input sequence and converts it into a fixed representation (context vector) using an RNN or LSTM.

- Processes input tokens sequentially and updates hidden states
- Captures relationships between words in the sequence
- Produces final hidden and cell states forming the context vector

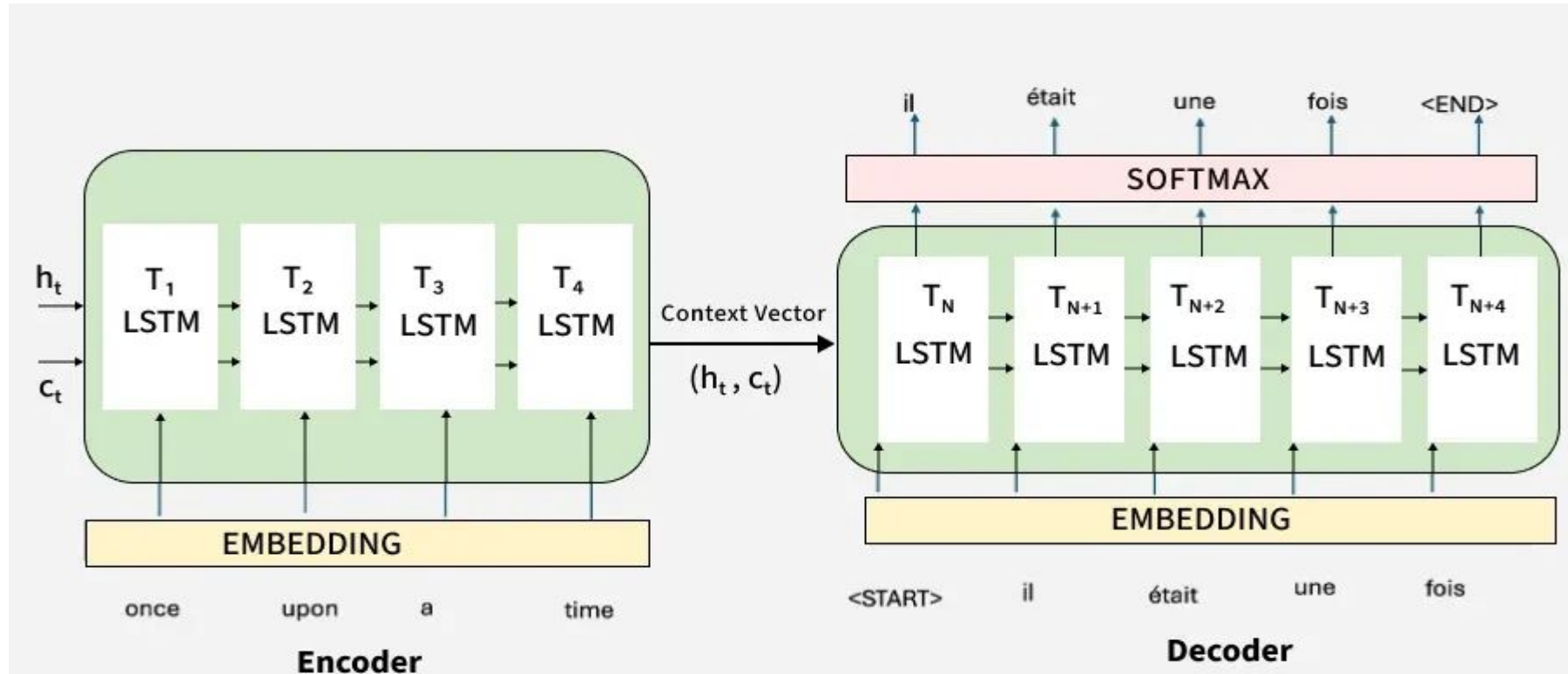


Decoder

The decoder uses the context vector from the encoder to generate the output sequence step by step.

- Takes previous output and context to predict the next token
- Generates output sequentially until an end token is reached
- Initializes its states using the encoder's final states

Working of Encoder Decoder Model



Step 1: Tokenizing the Input Sentence

- The sentence "I am learning AI" is first broken into tokens: ["I", "am", "learning", "AI"].
- Each word (token) is converted into a vector that a machine can understand. This process is called embedding.

Step 2: Encoding the Input

- The encoder processes these embeddings sequentially using an LSTM network.
- At each step, it updates its hidden state based on the current word and previous context. This helps the model understand the sequence order and relationships between words.
- After processing the full sentence, the encoder generates a context vector (final hidden and cell states), which represents the meaning of the entire input sentence.

Step 3: Passing the Context to the Decoder

- The Context Vector is passed to the Decoder as shown in image.
- It acts like a summary of the full input sentence.

Step 4: Decoder Generates Output Step-by-Step

- The Decoder uses the context and starts creating the output one word at a time.
- First it predicts the first word then uses that to predict the second word and so on.

Step 5: Attention Mechanism

- Basic encoder-decoder uses a single context vector, which can limit performance for long sequences.
- Attention mechanism helps the decoder focus on different parts of the input at each step.
- Improves accuracy by not relying only on one fixed representation.

Step 6: Producing the Final Output

- The decoder continues generating until the full translated sentence is produced.
- Each output token depends on the previous ones and the input context. You finally see the output tokens generated on the right side of the diagram completing the translation.

Attention Mechanisms and Sequence Generation Tasks in Deep Learning

1. Introduction

Many deep learning applications involve **sequential data**, where the order of elements is important. Examples include:

- A sentence: *“I am learning Deep Learning”*
- A speech signal represented over time
- A time-series sequence
- A DNA sequence
- A sequence of video frames

Traditional neural networks process inputs independently and therefore are not well suited for such tasks. **Recurrent Neural Networks (RNNs)**, **LSTMs**, and **GRUs** were introduced to process sequential information. However, these models can face difficulties when sequences become long.

Attention mechanisms solve an important limitation by allowing the model to focus selectively on the most relevant parts of the input sequence while generating each output.

Sequence Generation Tasks in Deep Learning

- Sequence generation produces an **ordered output sequence**.
- Output is generated **one word/token at a time**.
- Common tasks:
 - Machine Translation
 - Text Summarization
 - Chatbots
 - Image Captioning
 - Speech Recognition
- Types:
 - One-to-One
 - One-to-Many
 - Many-to-One
 - Many-to-Many

Example:

English Sentence → Encoder-Decoder Model → Translated Sentence

Encoder–Decoder Architecture

- **Encoder:** Processes the input sequence and creates hidden representations.
- **Decoder:** Generates the output sequence step by step.
- Traditional models use a single **context vector**:

$c = h_T$

- **Problem:** Important information may be lost for long sequences due to the fixed-size context vector.

Flow:

Input → Encoder → Context Vector → Decoder → Output Sequence

Attention Mechanism

What is Attention?

- Allows the model to **focus on relevant parts of the input** while generating each output.
- Generates a **different context vector at each decoding step**.
- Calculates:
 1. Attention scores
 2. Attention weights using Softmax
 3. Weighted context vector

$$e_{t,i} = \text{score}(s_{t-1}, h_i) \quad \alpha_{t,i} = \text{Softmax}(e_{t,i}) \quad c_t = \sum_i \alpha_{t,i} h_i$$

Example: While generating “*book*”, the model focuses more on the corresponding input word “*book*”.

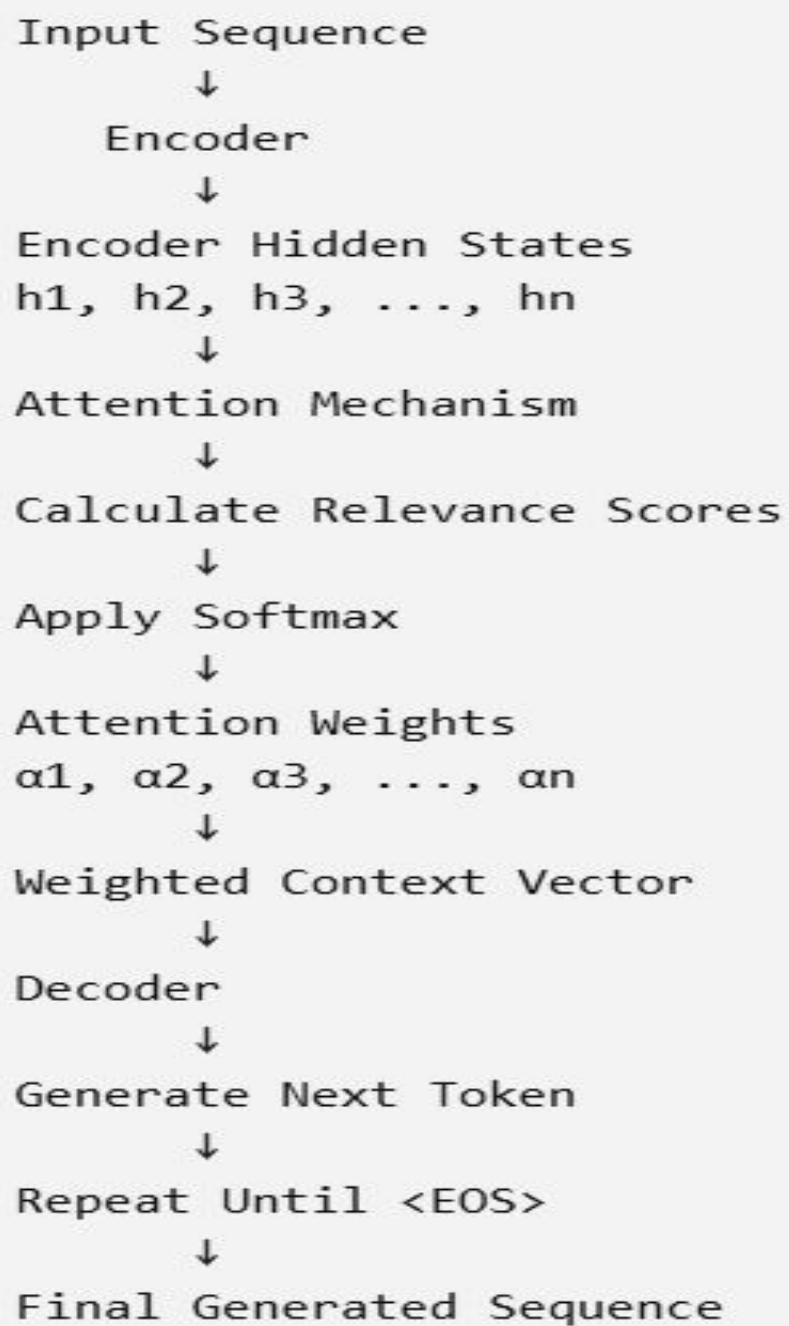
Types of Attention

- **Soft Attention:** Assigns importance to all input elements.
- **Hard Attention:** Focuses on one selected input element.
- **Global Attention:** Considers the complete input sequence.
- **Local Attention:** Focuses on a specific part of the sequence.
- **Self-Attention:** Elements attend to other elements in the same sequence.
- **Cross-Attention:** Output sequence attends to information from another sequence.

Modern Attention Formula

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

Where Q = Query, K = Key, V = Value.



Input Sequence

The student studied regularly for the examination and achieved excellent marks.

Output Sequence

Student achieved excellent marks.

This is a **sequence generation task** because the model reads one sequence and generates another sequence.

Long Input Text → Encoder–Attention–Decoder → Short Summary

Encoder Processes the Input

The encoder processes each word and creates hidden states.

The	→	student	→	studied	→	regularly	→	examination	→	achieved	→	excellent	→	marks
↓		↓		↓		↓		↓		↓		↓		↓
h_1		h_2		h_3		h_4		h_5		h_6		h_7		h_8

The hidden states contain information about the input words and their context.

$$h_1, h_2, h_3, \dots, h_8$$

Attention Focuses on Important Information

While generating the word “**Student**”, attention focuses mainly on:

Input Word	Attention Weight
The	0.02
student	0.75
studied	0.05
regularly	0.03
examination	0.02
achieved	0.05
excellent	0.04
marks	0.04

The attention mechanism calculates a context vector:

$$c_t = \sum_i \alpha_{t,i} h_i$$

The highest attention is given to the most relevant word.

Attention Changes for Each Generated Word

The decoder generates the summary **one word at a time**.

Generated Output

Student
Achieved
Excellent
Marks

Main Attention Focus

student
achieved
excellent
marks

Input:

student → studied → examination → achieved → excellent → marks



ATTENTION



Decoder:

<START> → Student → achieved → excellent → marks → <EOS>

At every decoding step, the model calculates **new attention weights**.

Complete Start-to-End Process

Step 1: Input

The student studied regularly for the examination and achieved excellent marks.



Step 2: Encoder

Input Words $\rightarrow h_1, h_2, \dots, h_8$



Step 3: Attention

The model identifies important words:

student, achieved, excellent, marks



Step 4: Decoder Generates Sequence

$\langle \text{START} \rangle \rightarrow \text{Student} \rightarrow \text{achieved} \rightarrow \text{excellent} \rightarrow \text{marks} \rightarrow \langle \text{EOS} \rangle$

Final Generated Summary

Student achieved excellent marks.

Key Idea: Attention helps the model focus on the most important parts of the input while generating each word of the output.

Feature	RNN	LSTM	GRU	Bidirectional Models	Encoder–Decoder
Full Form	Recurrent Neural Network	Long Short-Term Memory	Gated Recurrent Unit	Bidirectional RNN/LSTM/GRU	Sequence-to-Sequence Architecture
Main Purpose	Process sequential data	Learn long-term dependencies	Learn long-term dependencies efficiently	Use information from both directions	Convert one sequence into another
Memory	Simple hidden state	Cell state + hidden state	Hidden state	Forward + backward hidden states	Context vector/hidden representations
Gates	No gates	Input, Forget, Output gates	Update and Reset gates	Depends on RNN/LSTM/GRU used	May use attention mechanisms
Long-Term Dependency	Weak for long sequences	Very good	Very good	Improved contextual understanding	Depends on encoder and decoder
Vanishing Gradient Problem	High	Reduced significantly	Reduced significantly	Depends on underlying model	Reduced when LSTM/GRU is used
Computational Complexity	Low	High	Medium	Higher due to two directions	Depends on architecture
Training Speed	Fast	Slower	Faster than LSTM	Slower than unidirectional models	Depends on sequence length
Parameters	Fewer	More	Fewer than LSTM	Approximately double underlying model	Depends on encoder and decoder
Processing Direction	Forward only	Forward only	Forward only	Forward + Backward	Input sequence → Output sequence
Best Use	Short/simple	Long and complex	Long sequences with	Tasks requiring past and	Translation.

1. Explain the structure and working of a biological neuron. How is it related to an artificial neuron?
2. Explain the Perceptron model with its architecture, mathematical equation, working, and limitations.
3. Explain Multilayer Feed-Forward Neural Networks. How do they overcome the limitations of a single-layer Perceptron?
4. Explain Forward Propagation and Backpropagation algorithms used for training a neural network with suitable examples.
5. Compare the following activation functions: Linear, Sigmoid, Tanh, Hard Tanh, ReLU, and Softmax. Mention their formulas, ranges, advantages, and applications.
6. What is a loss function? Explain the notation of a loss function and discuss important loss functions used for regression problems.
7. Explain loss functions used for classification problems, including Binary Cross-Entropy and Categorical Cross-Entropy.
8. What are hyperparameters in a neural network? Explain the role of Learning Rate, Regularization, Momentum, and Sparsity in neural network training.
9. Explain Gradient Descent and its variants. How does the learning rate affect the convergence of a neural network?
10. What are the Vanishing Gradient and Exploding Gradient problems? Explain their causes, effects on neural network training, and possible solutions.